

上下文工程实战指南：让 Agent 少犯蠢的工程方法论

👤 Guide 📁 AI 应用开发 📖 约 6270 字 ⌚ 大约 21 分钟

大家好，我是 Guide。

这两年 AI 圈有个特别有意思的现象：同样的模型、同样的代码框架，为什么别人的 Agent 能稳稳当当完成任务，你的却动不动就迷失方向、重复操作、或者输出一些看起来很对但实际跑不通的东西？

答案大概率出在上下文上。

从一个例子说起

为什么同样的模型，Agent 表现却天差地别？

先看一个电商售后场景。用户发来一条消息：

“我上周买的耳机右耳没声音了，怎么处理？”

简陋版 Agent（上下文贫瘠）：

- 1 User：我上周买的耳机右耳没声音了，怎么处理？
- 2 Model：抱歉给您带来不便。请问您购买的是哪款耳机？订单号是多少？能否描述一下具体故障表现？

代码逻辑完全正确，LLM 调用也正常，但输出像个翻流程手册的客服新人——永远在要信息，从不主动整合。

丰富版 Agent（上下文充足）：

在调用 LLM 之前，系统先做了一轮上下文组装：

- 查订单系统 → 定位到上周的购买记录：索尼 WH-1000XM5，3 月 25 日下单
- 查保修状态 → 还在 7 天无理由退换期内
- 查用户历史工单 → 该用户是老客户，之前无售后纠纷
- 挂载 create_return_order 和 check_inventory 工具

然后才生成回复：

“您好，查到您 3 月 25 日购买的索尼 WH-1000XM5，目前还在退换期内。我这边直接帮您发起换货申请，仓库显示同款有库存，预计 2-3 天寄出新品。需要我操作吗？”

上下文的质和量变了。

一句话：当前 **Agent** 的大部分失败，根源在上下文。上下文不够，模型再强也没用；上下文对了，中等水平的模型也能完成任务。

理解 Context Engineering

它和 Prompt Engineering 到底有什么区别？

Tobi Lutke 有句话说得特别到位：Context Engineering 是 "the art of providing all the context for the task to be plausibly solvable by the LLM"——给 LLM 提供足够的上下文，让任务在它的能力范围内变得有可能被解决。

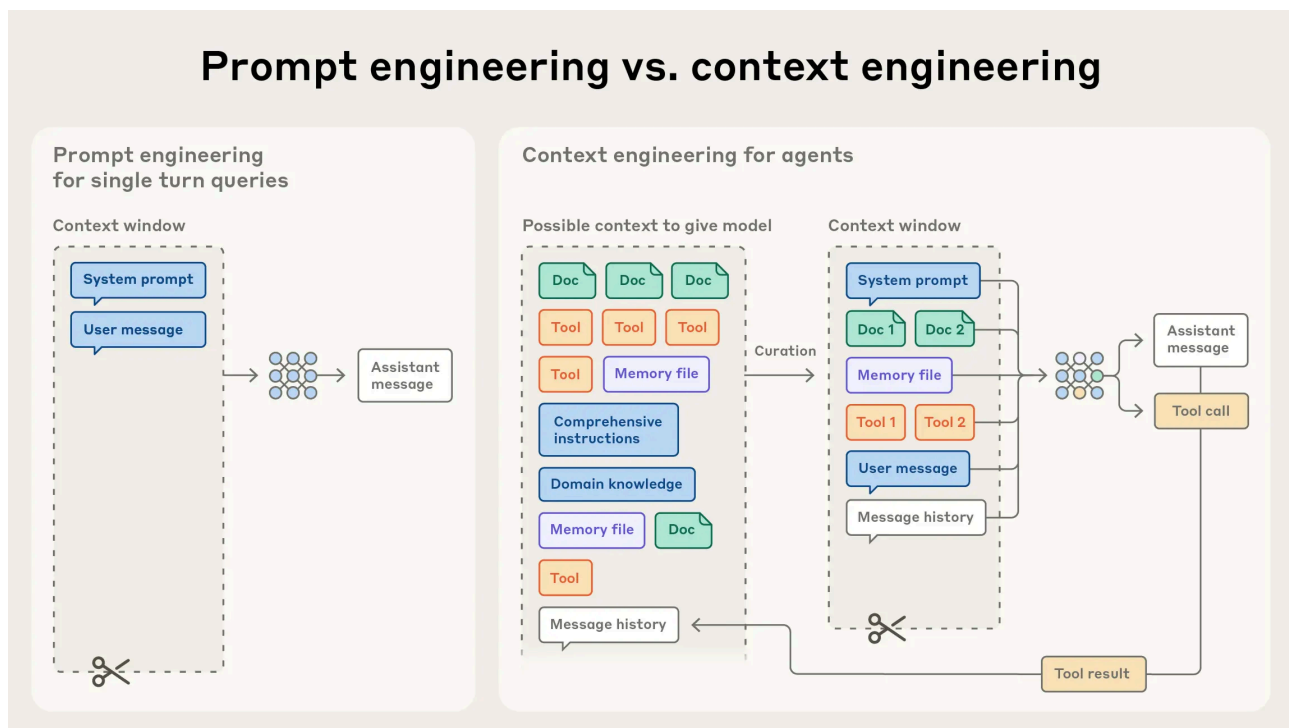
注意这里的关键词是 **plausibly**，强调的不是“LLM 一定能解决”，而是“有了足够上下文，任务才变得合理地可解”——这是一种对模型能力边界的谨慎预期。

很多文章把 Context Engineering 和 Prompt Engineering 混为一谈，这是不对的。

- **Prompt Engineering** 聚焦于指令本身的撰写和组织编排，核心问题是“怎么措辞、怎么排列”。
- **Context Engineering** 是构建一套动态系统，核心问题是“什么信息、以什么格式、在什么时机填入上下文”。

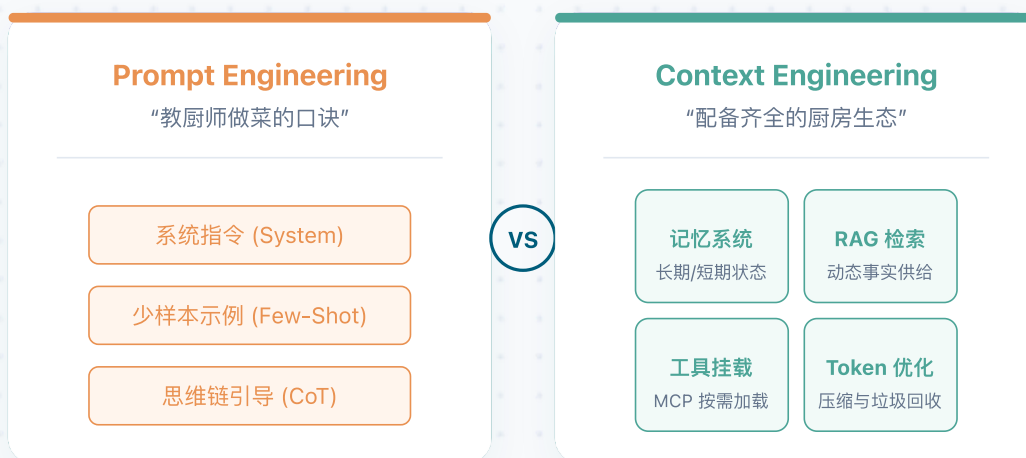
这张图是 Anthropic 官方博客中的，非常形象地对比了二者：

Prompt engineering vs. context engineering



如果说 Prompt Engineering 是教厨师做菜的一句口诀，那 Context Engineering 就是给他一间配备齐全的厨房——包括食材储备、刀具分类、火候参考手册。

Prompt vs Context 工程维度对比



公众号: [JavaGuide](#)
网站: [javaguide.cn](#)

换个角度理解：**Context Engineering 就是 LLM 的“内存管理与页面置换”。**

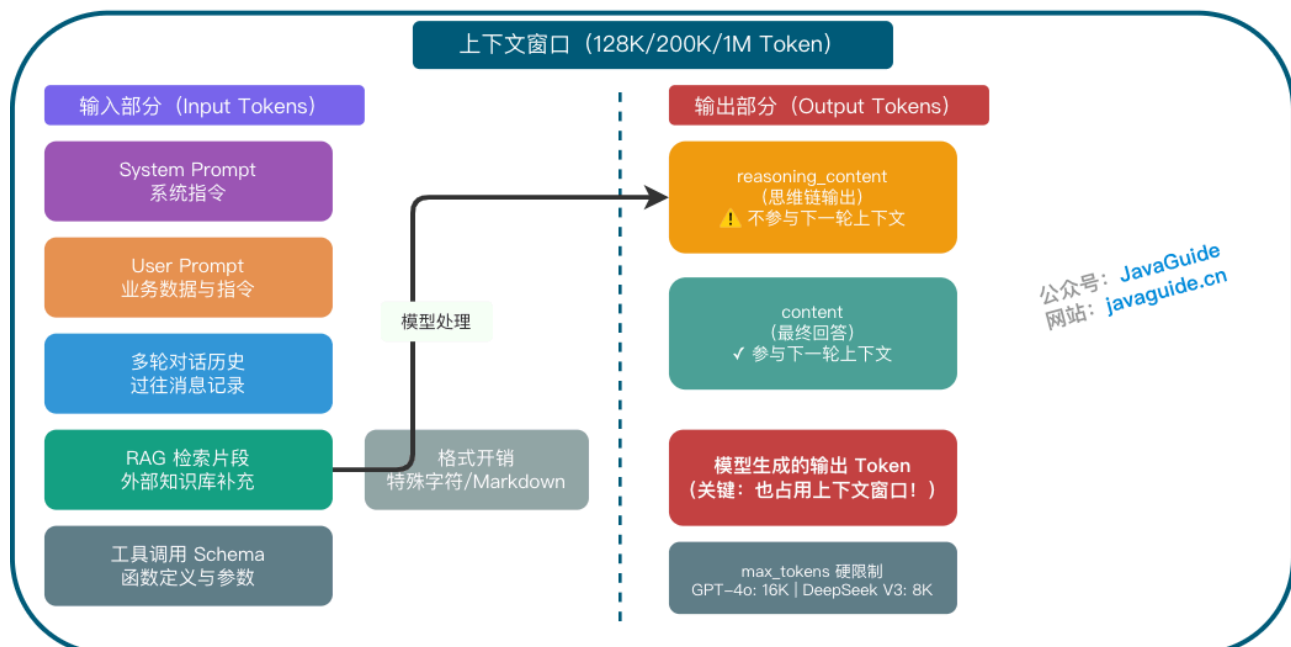
LLM 的上下文窗口是有限的内存，Context Engineering 决定了这块内存里装什么、换出什么、什么时候读写。当上下文窗口满时，需要决定淘汰哪些内容——这和操作系统页面置换算法（LRU、优先级策略）的思路完全一致，也正好对应后面要讲的三层 Token 降级策略。

Context Engineering 具体包含哪些内容？

从实战角度，Context Engineering 管的事情可以分为六大核心板块：

- **System Prompt（系统指令）**：静态 Prompt 的结构化编排。比如 `.cursorrules`、`.claude/rules` 这类配置文件，核心是把角色设定、目标、约束、执行流、输出格式拆解清楚，让模型在复杂任务里不脱轨。
- **User Prompt**：业务数据与指令。
- **Memory（记忆系统）**：短期记忆（Session 滑动窗口管理）和长期记忆（核心事实提取 + 向量数据库存储）。
- **RAG & Tools（动态增强）**：按需检索外部文档作为背景知识 + 把工具描述以结构化形式挂载到上下文。本质上，RAG 就是 Context Engineering 的一种特定实现模式——“检索什么、怎么检索、检索结果怎么填入上下文”这三个问题，本身就是上下文工程。
- **Structured Output（结构化输出）**：输出格式的定义，比如 JSON Schema、function call 的返回结构等。这直接影响下游消费方的解析和后续 Agent 链路的衔接，是容易被忽视但实战价值很高的一环。
- **Token 优化（上下文裁剪）**：摘要压缩、历史剔除、Context Caching，在保证信息完整度的同时控制 Token 消耗。

上下文窗口（Context Window）= LLM 的「工作记忆」



核心技术板块

如何做好静态规则的结构化编排？

这是 Agent 的“出厂设置”。

业界主流做法是用高度结构化的 Markdown 格式编排系统提示词，强制划分出：[Role] 角色设定、[Objective] 核心目标、[Constraints] 严格约束、[Workflow] 标准执行流、[Output Format] 输出格式。

一个典型的工程实践：

```
1  ## 角色
2  你是一个后端服务故障排查专家，擅长通过日志和监控数据定位问题根因。
3
4  ## 约束
5  - 只调用必要的工具，不重复调用相同逻辑的工具
6  - 发现关键信息时立即停止搜索，输出结论
7  - 优先使用实时数据而非历史推断
8
9  ## 执行流
10 1. 查监控指标（CPU/内存/网络）
11 2. 查对应时间范围的日志
12 3. 如发现异常调用链，追踪上下游依赖
13 4. 输出结构化报告：问题描述 → 根因 → 建议修复方案
14
15 ## 输出格式
16 使用 JSON，包含字段：incident_summary, root_cause, evidence,
    recommendation
```

把这些规则固化为 `.cursorrules` 或 `AGENTS.md` 文件，Agent 在复杂任务里的“脱轨”概率会大幅降低。值得一提的是，随着模型能力不断提升，Prompt 格式的精确性可能正在变得不那么关键——但结构化编排带来的**可维护性**和**团队协作效率**提升是长期价值。

动态信息应该怎样按需挂载？

上下文窗口不是垃圾桶，不能什么信息都往里塞。要做到精准挂载，至少有两个关键切入点：

- 工具的懒加载 (Tool Retrieval)**：当 Agent 面对大量 MCP 工具时，一股脑全部挂载会直接撑爆上下文并增加误调用概率。一种可行的工程方案是：先通过向量检索选出当前任务最相关的 Top-5 工具定义，按需挂载——这和人类专家面对新问题时翻手册找相关章节是一个逻辑。当然，Anthropic 更强调的是在设计阶段就精简工具集，避免工具集合过度膨胀导致决策模糊。
- 动态记忆与 RAG**：短期记忆通过滑动窗口管理，长期事实通过向量数据库检索。每次挂载前，LLM 还要对 Observation（如 API 返回的报错日志）做一次“摘要提炼”，只把核心结论写回上下文，而非原始数据洪流。

Token 预算不够用时如何降级？

这是复杂工程里的核心挑战。当长任务接近上下文窗口极限时，必须有优先级剔除策略：



优先级	内容	处理方式
低优先级（可折叠）	早期对话历史	AI 摘要压缩
中优先级（可精简）	RAG 检索的背景资料	二次裁剪，保留核心段落
高优先级（绝对保护）	System Constraints、当前核心工具描述	永不丢失，确保逻辑一致性

配套优化手段是 **Context Caching**：在大规模并发请求里，相同 System Prompt 部分只需加载一次，显著降低首 Token 延迟和推理成本。

上下文失效的根因

为什么上下文越长，效果反而可能越差？

很多人在使用超长上下文模型时会有个误解：上下文越长，模型能用的信息越多，效果应该越好。

错了。真实情况是：**上下文存在边际效益递减，甚至可能负向增长。**

背后的原因是 LLM 的 Attention 机制。Transformer 架构让每个 Token 都要和上下文里所有其他 Token 计算注意力关系，这意味着 n 个 Token 的上下文会产生 n^2 量级的注意力计算。

当上下文从 1K 扩展到 100K Token，并非“均匀稀释”那么简单。真正的问题是：**模型在更多 token 间区分“相关”与“不相关”的辨别力下降**。Softmax 注意力每个 query token 的权重之和恒为 1，上下文变长后， n^2 量级的 pairwise 关系让精确捕捉长程依赖变得更困难——信噪比越低，模型越难从噪声中挑出信号。这就是“Context Rot”（上下文腐化）现象——随着上下文 Token 总量增大，模型整体的信息回忆能力随之下降。与之相关的还有学术界发现的 **Lost in the Middle** 问题：模型对位于上下文中间位置的信息记忆力显著低于开头和结尾，呈 U 型分布。两者共同说明了一个事实：上下文并非“越长越好”。

更关键的是，模型的 Attention 模式是在短序列数据上训练出来的——互联网文本的平均长度远低于现在的上下文窗口。这意味着模型处理长依赖关系时没有足够的学习经验，位置编码的外推能力也有限。虽然有位置编码插值技术（Position Encoding Interpolation，如基于 RoPE 的 YaRN、NTK-aware Interpolation 等）来缓解长序列外推问题，但精度损失是结构性的，不会完全消失。

工程启示：不同模型的衰减曲线不同——有些模型的退化比较平缓，有些则比较陡峭，因此上下文长度的最优阈值需要针对具体模型实测。但有一点是确定的：上下文必须被当作有限资源来管理，不是塞满越好。找到“高信噪比”的平衡点，是 Context Engineering 最核心的手艺。

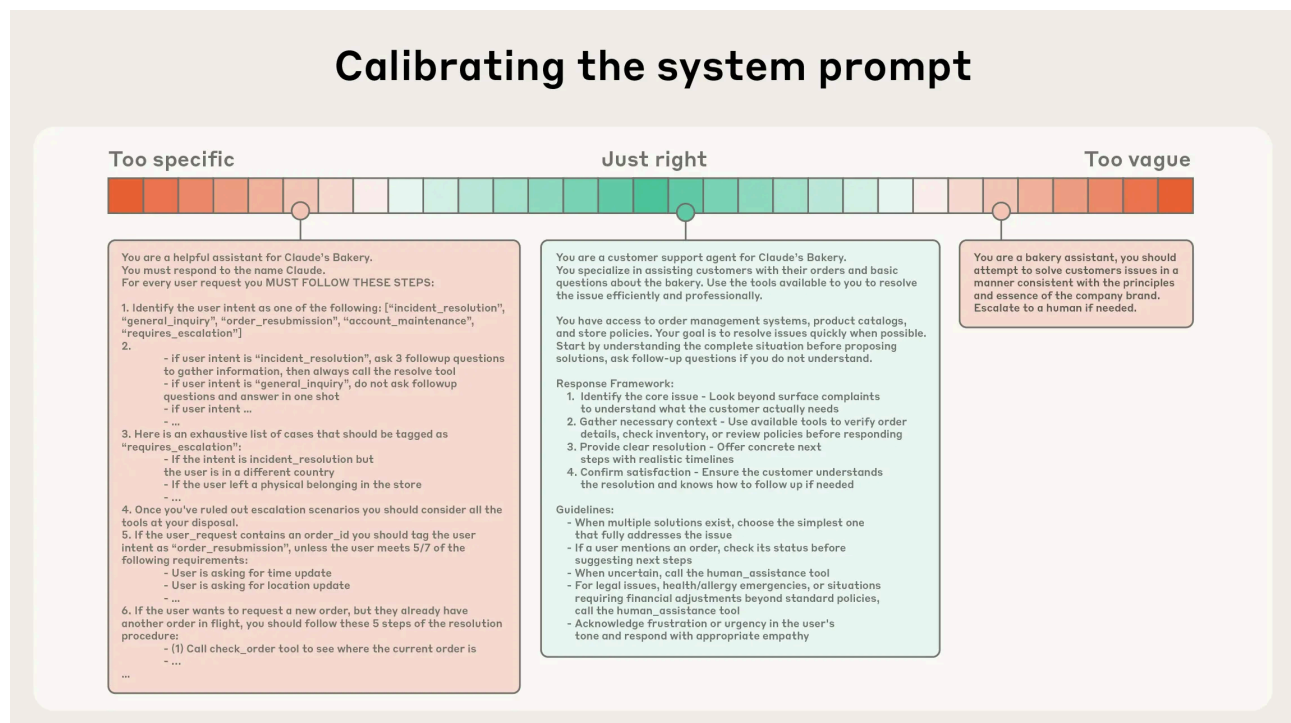
有效上下文的构建原则

System Prompt 怎样写才算“恰到好处”？

System Prompt 的编写存在两个常见失败模式：

- **第一个极端：过度设计。**工程师把复杂的 if-else 逻辑硬编码进 Prompt 里，试图精确控制 Agent 的每一步行为。结果是指令脆弱得像纸片房，维护成本极高，而且模型在未见过的边缘情况里依然会脱轨。
- **第二个极端：过度抽象。**只给“你要做一个有帮助的助手”这种模糊指令，模型无法从中获得足够的决策依据，要么频繁追问用户，要么输出与业务预期严重偏离。

正确的做法是：**足够具体以引导行为，同时足够抽象以提供通用启发。**具体和抽象之间的平衡点，就是 Anthropic 工程博客中提到的"Goldilocks zone"（刚刚好的区域）。



一个实操建议：先用最小化的 Prompt 测基线表现，然后基于 failure case 逐条补充清晰指令。不要在第一天就试图穷举所有规则。

工程提示：Anthropic 的做法是"Calibrating the system prompt"——把 System Prompt 当成一个需要持续调校的参数，而不是一次性写死的产品配置文档。每发现一个 failure case，针对性地加一条清晰规则，然后重新测试。

工具描述如何设计才不会误导 Agent?

工具定义的质量直接决定 Agent 是否“选对武器”。

好的工具描述需要明确回答两个问题：**什么时候该调用和什么时候不该调用。**如果一个工具的描述让人类工程师都无法判断该不该用，Agent 肯定也会犯错。

常见失败案例是“大而全”的工具——把一堆相关但各自独立的功能塞进一个工具里，比如 `manage_database` 同时包含“建表、查数据、删数据、备份、导出”五个能力。Agent 在选择工具时会陷入模糊判断，在填充参数时也会被无关字段干扰。

 **常见误区：**很多人觉得工具描述写得越详细越好。实际上，工具描述的关键在于“边界清晰”而非“面面俱到”——什么时候该用、什么时候不该用，这两条线划清楚，比堆砌功能描述有效得多。

一个工具只做一件事，参数描述要包含格式示例。这是工程化的基本准则，也是 Agent 工具设计的核心原则。

Few-shot 示例应该怎么选、选几个？

Few-shot prompting（给示例）是经过验证的有效策略，但很多人用错了。

典型错误是往 Prompt 里塞几十个 edge case 示例，试图覆盖所有规则。这种做法的问题是：模型会过度拟合这些示例的表层模式，而忽略真正应该学的底层逻辑。

业界常用的做法是选 **3-5 个多样化的典型示例（canonical examples）**。Anthropic 也强调了示例的多样性和典型性比数量更重要——“Canonical”的意思是“权威的、标准化的”，每个示例要能代表一类典型场景的解决模式，而非覆盖所有边缘情况。对模型来说，示例是“一幅画胜千言”的可视化教学，展示“什么情况用什么策略”而非“什么输入对应什么输出”。

运行时上下文检索

为什么预检索在复杂 Agent 场景下不够用？

传统 AI 应用的做法是**预检索**：在调用 LLM 之前，先通过 Embedding 相似度把最相关的上下文全部找出来，一股脑塞进 Prompt。

这套机制在简单场景下工作良好，但在 Agent 化的复杂任务里开始暴露问题：预检索拿到的信息是“静态相关”的，但 Agent 在执行过程中会动态发现新线索，而这些新线索在预检索时根本不存在。

Just-in-Time 按需加载是怎么工作的？

Just-in-Time（按需加载） 策略因此兴起。

其核心思想是：Agent 运行时不要预先装载所有可能相关的信息，而是维护轻量级的引用句柄（文件路径、存储查询、Web 链接），在真正需要时才通过工具动态拉取数据。

拿 Claude Code 举例：它处理大数据库分析时，不是把所有数据 Load 进上下文，而是写定向查询语句、存储结果、用 `head / tail` 命令分析数据文件。Agent 像人类一样通过“文件名”和“目录结构”理解信息位置，通过“文件大小”和“时间戳”判断重要性，而不是一开始就加载全部内容。

这种策略还有额外好处：**元数据本身就是信息**。`tests/test_utils.py` 和 `src/core_logic/test_utils.py` 的语义差异靠文件路径就传递了，不需要额外解释。Agent 能从上下文结构中提取意图，这是一种接近人类认知的高效方式。

Anthropic 把这种方式称为**渐进式披露 (Progressive Disclosure)**：Agent 通过层层探索逐步构建对信息的理解，而不是一次性获取全部上下文。每一次交互都揭示新的上下文，进而引导下一步决策——文件大小暗示复杂度，时间戳代表相关性，目录结构传递语义。

当然，按需加载有明显的代价：**运行时探索比预检索更慢**，而且需要工程师提供足够好的导航工具（`glob`、`grep`、`tree` 等）让 Agent 能在信息海洋里不迷路。

 **常见误区**：很多人以为 Just-in-Time 就是“不预处理就好了”。实际上恰恰相反——按需加载对工具集和导航策略的设计要求更高。如果导航启发式规则不够好，Agent 容易误用工具、追入死胡同，浪费宝贵的上下文空间。

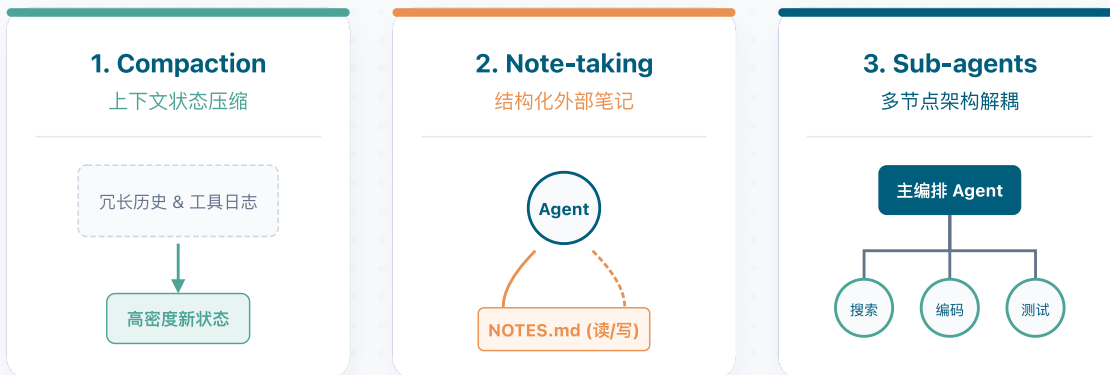
更重要的是，如果缺乏精心设计的导航启发式规则，Agent 容易陷入**探索失败模式**：误用工具、追入死胡同、错过关键信息。这些失败会直接消耗宝贵的上下文空间，让原本就有限注意力预算雪上加霜。所以 Just-in-Time 不是“不预处理就好了”，而是需要同时设计好工具集和导航策略。

最优解往往是混合策略：对确定性高的静态知识预检索，对动态发现的信息按需拉取。Claude Code 就是典型——`CLAUDE.md` 文件预加载，但具体的文件内容靠 Agent 运行时探索。

混合策略的决策边界也有规律可循：**动态内容占比高、探索空间大的场景**（如代码库分析、信息检索）适合 Just-in-Time 为主；**动态内容少、上下文稳定的场景**（如法律文书审阅、财务报表分析）更适合预检索 + 少量运行时补充。

长任务的上下文持久化

长任务上下文持久化：抵抗腐化的三大武器



上下文快满了怎么办？—— Compaction

当 Agent 需要连续工作数小时、处理数轮迭代时，单纯的上下文管理已经不够用，必须引入**跨窗口持久化机制**——上下文也需要像生物体一样具备新陈代谢能力，才能在长时间运行中保持有效。

Compaction（压缩） 就是第一种武器。

当上下文窗口快满时，把历史内容交给 LLM 总结，然后用摘要创建一个新的上下文窗口继续工作。Claude Code 的实现逻辑是：把历史消息传给模型做摘要，保留架构决策、未解决的 Bug、关键实现细节，丢弃冗余的工具调用结果。Agent 拿着这个压缩后的上下文加上最近访问的 5 个文件，继续工作。

难点在选择：保留太多则压缩无效，保留太少则关键上下文丢失。一个工程建议是：用复杂 Agent 轨迹数据反复调优你的压缩 Prompt——先最大化召回（不要漏掉重要信息），再逐步精简冗余内容。这是一个迭代调优的过程，而非一次性编写。

一个最轻量的压缩手段是**工具结果清理**：一旦工具在历史里被调用过且结果已被消化，后续上下文里这个结果的原始文本就没必要保留了。Anthropic 的 Developer Platform 已经把这个做成了原生功能。

工程提示：压缩 Prompt 的调优是个迭代过程。建议用复杂 Agent 轨迹数据反复调优——先最大化召回（不要漏掉重要信息），再逐步精简冗余内容。一次性编写完美的压缩指令几乎不可能，持续迭代才是正道。

如何让 Agent 学会“记笔记”？—— Structured Note-taking

Structured Note-taking（结构化笔记） 是第二种武器。

让 Agent 把关键进展以结构化格式写入外部文件（如 NOTES.md），后续基于新上下文重新读取。

这和人类工程师“写 to-do list 和技术备忘”的习惯完全一致。Claude Code 在长任务里会自动维护 to-do list，自定义 Agent 可以在项目根目录维护 NOTES.md ——包含当前进度、已知问题、下一步计划。

一个极端但令人印象深刻的案例是 **Claude 玩 Pokemon**：在数千轮游戏步骤里，Agent 自主维护了精确的数值追踪（“过去 1234 步我在 1 号道路训练皮卡丘，已升 8 级，距离目标还差 2 级”），还自发建立了地图、成就清单、战斗策略笔记。这些笔记在上下文重置后依然能被读取，使跨越数小时的游戏训练成为可能。

Anthropic 在 Sonnet 4.5 发布时推出了 Memory Tool 公开测试版，通过文件系统的持久化让 Agent 建立跨会话的知识库。

什么时候该把任务拆给多个 Agent？—— Sub-agent 架构

Sub-agent Architectures（多 Agent 架构） 是第三种武器。

不是让一个 Agent 维护整个项目的状态，而是让专业化的子 Agent 处理专门任务，主 Agent 只负责任务编排和结果汇总。

每个子 Agent 可以探索大量上下文（数万个 Token），但返回给主 Agent 的只是 1000-2000 Token 的高度浓缩摘要。这种设计实现了关注点分离：详细搜索上下文被隔离在子 Agent 内部，主 Agent 保持干净的上下文专注于分析和决策。

Anthropic 在 "How we built our multi-agent research system" 里详细描述了 this 模式，相比单 Agent 在复杂研究任务上实现了显著的质量提升。

三种技术怎么选：

技术	适用场景
Compaction	需要持续对话的长流程，保持上下文连贯性
Note-taking	迭代式开发、有清晰里程碑、多步推进的任务
Sub-agents	复杂研究、需要并行探索、结果需汇总的场景

工具链与工程落地

落地 Context Engineering 需要哪些工具？

说完方法论，顺手整理下工程落地需要的主流工具：

编排框架：LangChain、LangGraph 这一类框架负责 Agent 的控制流、状态管理和循环调度。

数据框架：LlamaIndex 专注 RAG 场景下的数据摄取、索引和检索优化。

向量数据库：Pinecone、Weaviate、Chroma、Qdrant 这一类负责 Embedding 的存储和语义搜索。

通信协议：MCP（Model Context Protocol）解决了“工具如何标准化接入宿主程序”的问题，被誉为 AI 领域的 USB-C。Anthropic 发布的 MCP 协议基于 JSON-RPC 2.0，定义了 Tools（可执行函数）、Resources（只读数据）、Prompts（可复用模板）三类标准原语。

Memory 产品：Memo、LETTA（原 MemGPT）、ZEP 这类专门做 Agent 记忆层的平台，在向量库之上封装了记忆写入、检索、遗忘的完整生命周期管理。

总结

Context Engineering 之所以重要，是因为它意味着工作重心的转移：**从优化单个 Prompt，到设计整个信息供给系统。**

过去我们关心的是“怎么措辞”，现在我们关心的是“构建什么样的上下文工程架构”。模型能力在增长，但注意力是有限的——这个基本约束不会因为模型变强就消失。

具体到工程实践，记住四条核心原则：

1. **上下文是系统输出，不是静态配置。**每次 LLM 调用前，你都在组装一个动态的上下文——这个组装逻辑本身才是工程的核心。
2. **高信噪比优于高信息量。**上下文的长度不决定效果，找到让模型做出正确决策所需的最小高密度信息集，才是手艺。
3. **上下文需要代谢机制。**对于长任务，没有什么是一“次组装永久有效”的——压缩、笔记、多 Agent 分层，这些机制让上下文在时间维度上保持新鲜和可用。
4. **从最简方案开始，逐步增加复杂度。**Anthropic 反复强调 “do the simplest thing that works”——先用最小可行的上下文方案跑通基线，再基于实际 failure case 逐层优化。过度工程化的上下文系统和不足的上下文一样危险。

Agent 失败的根源大多在上下文精度不够。把上下文工程做到位，中等水平的模型也能完成看似复杂的任务。

参考

- [Effective context engineering for AI agents - Anthropic](#)
- [Context Engineering: The New Frontier of AI Development](#)
- [The New Skill in AI is Not Prompting, It's Context Engineering](#)
- [Context Engineering by Simon Willison](#)
- [Own your context window](#)

最近更新: 2026/4/13 15:43

贡献者: SnailClimb, Kou Shuang, guide, Guide